For now, let’s implement the single-iteration gradient descent solution, starting with listing 16.5.

Listing 16.5 Method forceDirectedEmbedding

```
function forceDirectedEmbedding(graph, alpha, maxSteps)
                  #1
  for v in graph.vertices
do                                                    #2
    (x[v], y[v]) ← randomVertexPosition()
  for _ in {1..maxSteps} do
                                    #3
    for v in graph.vertices do
                   #4
      x1[v] ← x[v] – alpha * derivative(graph, v,
x)                  #5
      y1[v] ← y[v] – alpha * derivative(graph, v, y)
    if x == x1 and y == y1
then                                                  #6
      break
    (x,y) ← (x1,
y1)                                                          #7
  return (x,y)
```

#1 Method `forceDirectedEmbedding` takes a graph, a learning rate alpha, and the maximum number of steps to perform, `maxSteps`. It returns a point in the domain: an assignment for the coordinates of each vertex.

#2 Cycle through each vertex, assigning its position randomly.

#3 Starts the iteration, running the main cycle at most `maxSteps` times.

#4 Cycle, again, through all vertices in the graph.

#5 For each vertex, updates its x and y coordinates by using gradient descent rules. We must use a new variable to hold these new values, because, for gradient descent to work, we need to compute all the gradients using the coordinates at current iteration, before the update.